UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

CEF440: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

PROJECT: DESIGN AND IMPLEMENTATION OF A MOBILE APPLICATION FOR COLLECTION

OF USER EXPERIENCE DATA FROM MOBILE NETWORK SUBSCRIBERS

GROUP 2

FINAL PROJECT REPORT

S/N	NAME	MATRICULE
1.	EFUETNGONG GENESIS TAZISONG	FE23A041
2.	MATUTE MOLUA FRITZ	FE23A087
3.	NWANTOLY STEPHILL FAVOUR BENYELLA	FE23A127
4.	NYANYOH DIVINE-FORTUNE BANFILA	FE23A128
5.	TENKEN MANPLA ULRICH	FE23A170

COURSE INSTRUCTOR: Dr. Valery Nkemeni

2025 / 2026 ACADEMIC YEAR

ABSTRACT

Mobile network subscribers in Cameroon continue to experience inconsistent service quality, including signal degradation, high latency, and unstable connections, while network operators often rely on technical, network-side statistics that fail to capture the true Quality of Experience (QoE) perceived by end users. This project presents the design and implementation of QoETRACK, an Android mobile application that bridges this gap by combining passive, automated collection of network performance metrics with active, subjective user feedback.

The project followed a complete software development lifecycle beginning with a study of mobile application development approaches, followed by structured requirement gathering and analysis, formal specification through a Software Requirements Specification (SRS) document, system modelling using UML and structured-analysis diagrams, user interface design and frontend implementation in Kotlin, and finally database design and backend implementation using an offline-first Room/SQLite and cloud-hosted MySQL architecture connected through an Express.js REST API deployed on Railway.

This report documents each phase of the project in a dedicated chapter, presenting the methodology, artefacts produced, technical decisions taken, and verification evidence collected throughout development. The resulting system demonstrates a functional, end-to-end pipeline capable of collecting, storing, synchronising, and eventually presenting real Quality of Experience data gathered from a physical Android device in Buea, Cameroon.

Contents

CHAPTER ONE: GENERAL INTRODUCTION

1.1 Background to the Study

Mobile communication has become an essential part of everyday life. As mobile networks continue to expand and evolve, users increasingly expect reliable connectivity, fast internet speeds, stable calls, and excellent overall service quality. However, many mobile subscribers still experience poor network performance, call drops, slow internet access, latency issues, and unstable connections.

Traditional methods used by mobile network operators to monitor network performance primarily focus on technical network parameters and often fail to adequately capture the actual experiences of end users. This creates a gap between network statistics and the real Quality of Experience (QoE) perceived by subscribers. This project proposes the design and implementation of a mobile application capable of collecting both subjective and objective QoE data from mobile network subscribers in real time.

1.2 Purpose of the System

The purpose of the proposed system is to provide an efficient, automated, and user-friendly platform for collecting and analysing mobile network user experience data. The system is intended to collect real-time user feedback regarding network quality, monitor mobile network performance automatically, store and analyse Quality of Experience data, generate reports and statistical visualisations, and help mobile network operators improve service quality and support data-driven decision-making.

1.3 Scope of the Project

The scope of this project covers the design and implementation of an Android-based mobile application capable of running continuously in the background, collecting network performance parameters, gathering user satisfaction feedback, recording timestamps and location information, storing collected data in a local and remote database, and generating charts and analytical reports to support Quality of Experience evaluation. The project mainly targets mobile network subscribers and network operators within Cameroon, and focuses primarily on Android devices because of their widespread use among mobile subscribers.

1.4 Objectives of the System

1.4.1 Main Objective

To design and implement a mobile application that enables real-time collection and analysis of user experience data from mobile network subscribers.

1.4.2 Specific Objectives

- Collect user satisfaction ratings regarding network quality.
- Monitor network performance automatically in the background.
- Record network-related parameters such as signal strength, latency, jitter, and bandwidth.
- Store data securely in both a local and a remote database.
- Generate statistical reports and visual charts.
- Assist mobile network operators in network optimisation.
- Provide a user-friendly interface for subscribers.

1.5 Stakeholders of the System

The system involves four broad categories of stakeholders. Mobile Network Subscribers are the primary users who provide subjective feedback and whose devices passively generate objective network metrics. Mobile Network Operators such as MTN Cameroon, Orange Cameroon, and Camtel are the intended consumers of the aggregated QoE data, using it to optimise coverage, detect faults, and improve customer satisfaction. System Developers are responsible for designing, implementing, and maintaining the application. Regulatory Authorities ensure that data collection and processing comply with telecommunications and data-privacy regulations.

1.6 Organization of the Report

This report is organised into nine chapters. Chapter One presents the general introduction to the project. Chapter Two reviews the mobile application development process, covering application types, languages, frameworks, and architectures. Chapter Three describes the requirement gathering process. Chapter Four presents the requirement analysis. Chapter Five presents the full Software Requirements Specification. Chapter Six presents the system modelling and design using UML diagrams. Chapter Seven documents the user interface design and frontend implementation. Chapter Eight documents the database design, backend implementation, and end-to-end synchronisation. Chapter Nine concludes the report with a summary of achievements, limitations, and recommendations for future work.

CHAPTER TWO: MOBILE APPLICATION DEVELOPMENT PROCESS

Before beginning design work, the group reviewed the general mobile application development process in order to make informed technology choices for the QoETRACK project. This chapter presents the findings of that review, covering the types of mobile applications, programming languages, development frameworks, architectures and design patterns, the requirement engineering process, and cost estimation.

2.1 Types of Mobile Applications

2.1.1 Native Applications

Native applications are developed specifically for a particular operating system using platform-specific languages and tools — Java or Kotlin within Android Studio for Android, and Swift or Objective-C within Xcode for iOS. Because they are designed for a single platform, native applications can fully utilise device hardware and operating system features such as the camera, GPS, sensors, notifications, and background services, resulting in high performance and a highly optimised, responsive user interface. Their main limitation is that separate codebases must be built and maintained for each platform, increasing development cost and time. Native applications remain the preferred choice when performance, security, and user experience are critical, which is why this approach was adopted for QoETRACK.

2.1.2 Web Applications

Web applications run inside a browser and are built using HTML, CSS, and JavaScript. They are accessed through a URL and require no installation, offering cross-platform compatibility at the cost of limited access to device hardware and generally lower performance.

2.1.3 Hybrid Applications

Hybrid applications combine web and native features, using web technologies wrapped in a native container through frameworks such as Ionic or Apache Cordova. They allow a single codebase to target multiple platforms and can access some device features through plugins, though generally with lower performance than fully native applications.

2.1.4 Progressive Web Applications (PWAs)

PWAs are advanced web applications that provide an app-like experience using standard web technologies. They support offline functionality through service workers and can be installed without going through an app store, but still have limited access to some device features compared to native applications.

2.2 Mobile App Programming Languages

Android applications are primarily developed using Java or Kotlin, with Kotlin now the preferred choice due to its modern, concise syntax and reduced potential for coding errors — the language used for QoETRACK. iOS applications are built using Swift or Objective-C. Cross-platform development uses JavaScript with React Native, Dart with Flutter, or C# with Xamarin, allowing a single codebase to target both major platforms at a slight performance cost.

2.3 Mobile App Development Frameworks

Native frameworks such as the Android SDK and iOS SDK give full access to device features and ensure high performance. Cross-platform frameworks — Flutter (Dart), React Native (JavaScript), and Xamarin (C#) — reduce development time and cost by sharing a single codebase across platforms. Hybrid frameworks such as Ionic and Apache Cordova use web technologies wrapped in a native container, trading some performance for development speed.

2.4 Mobile App Architectures and Design Patterns

Two broad architectural styles were considered. A Monolithic Architecture combines UI, business logic, and data handling into a single unit, which is simple but harder to scale. A Client-Server Architecture, adopted for QoETRACK, separates the mobile client from a remote server, supporting scalability, real-time updates, and efficient data management.

Three design patterns were reviewed: MVC (Model-View-Controller), which separates data, UI, and logic but can grow complex in large applications; MVVM (Model-View-View Model), which introduces a View Model to manage UI-related data and supports better data binding; and MVP (Model-View-Presenter), which improves testability by isolating logic in a Presenter. These patterns improve scalability, maintainability, and testability of the resulting application.

2.5 Requirement Engineering Process

The requirement engineering process followed for this project consisted of four stages: Requirement Gathering, using interviews and surveys to collect stakeholder needs; Requirement Analysis, categorising requirements into functional and non-functional; Requirement Specification, formally documenting requirements in a Software Requirement Specification (SRS); and Requirement Validation and Management, reviewing requirements for accuracy, completeness, and feasibility, and managing changes throughout development. Each of these stages is documented in the chapters that follow.

2.6 Mobile App Development Cost Estimation

Mobile application development cost is influenced by several factors: the complexity of required features, the target platform(s), the range of functionalities implemented, the size and experience of the development team, and ongoing maintenance needs. Estimation techniques considered include Expert Judgment, based on developer experience; Analogous Estimation, based on comparable prior projects; and Parametric Estimation, which uses measurable parameters such as time and resources. Cost categories typically span development, testing, deployment, and maintenance.

2.7 Chapter Summary

This chapter has reviewed the landscape of mobile application development, providing the technical justification for the choices made in this project: a native Android application built with Kotlin, following a client-server architecture with an offline-first local database and a cloud-hosted backend. These decisions directly shaped the requirement gathering, analysis, and design work presented in the subsequent chapters.

CHAPTER THREE: REQUIREMENT GATHERING

Requirement gathering is a critical phase in system development, as it defines what the system must achieve and how it should function. This chapter documents the stakeholder identification, techniques employed, and results of the requirement gathering process carried out for the QoETRACK project.

3.1 Stakeholder Identification

3.1.1 Primary Stakeholders — Mobile Network Subscribers

Mobile network subscribers are the direct users of the application and the main source of collected data. They interact with the system actively, by rating call clarity, internet speed, call-drop frequency, and overall satisfaction, and passively, through automatic collection of signal strength, network type, latency, bandwidth, location, and timestamp data. Their participation determines both the volume and reliability of collected data, so the application had to be designed to be user-friendly, non-intrusive, secure, and beneficial to the user.

3.1.2 Secondary Stakeholders

- Mobile Network Operators (MTN, Orange, Camtel) use collected data to optimize resource allocation, detect faults quickly, improve reliability, and reduce churn.
- System Developers responsible for designing, implementing, testing, and maintaining the application, ensuring efficiency, security, and scalability.
- Regulatory Authorities oversee data privacy, security, and compliance with national and international data-protection laws.

3.2 Requirement Gathering Techniques

Multiple techniques were combined to capture both quantitative and qualitative insights.

3.2.1 Surveys

A structured survey was developed using Google Forms and distributed to mobile network subscribers, collecting 133 responses covering age range, occupation, network provider, operating system, network performance ratings, call-drop experiences, and location of network usage. The survey served as the primary quantitative data collection tool.

3.2.2 Interviews

Interviews were conducted with a small group of subscribers representing different demographics to obtain qualitative insight into user frustrations, expectations, and feature preferences. The interviews revealed that

many users experience inconsistent performance during peak hours, that call drops and slow internet are common complaints, and that users expressed strong interest in an application capable of monitoring network performance in real time and helping them make better network choices.

3.2.3 Brainstorming

Group brainstorming sessions generated candidate features for the system, including real-time network performance monitoring, automatic data collection of signal strength, speed and latency, a user feedback rating system, an alert/notification system for poor conditions, a data-visualisation dashboard, and location-based performance tracking.

3.2.4 Reverse Engineering

Existing network monitoring applicationsSpeedtest, Opensignal, and nPerfwere reviewed to identify best practices and avoid reinventing existing solutions. Useful features identified included real-time display of signal strength and network type, internet speed testing, simple user interfaces, graphical representation of metrics, and basic activity logging. These informed several design decisions in the QoETRACK dashboard.

3.3 Data Gathering and Cleaning

A total of 133 survey responses were collected, comprising subjective data (network experience ratings, call-drop occurrences, overall satisfaction) and objective data (age group, occupation, operating system, network provider, and location of usage). The raw dataset was cleaned in Microsoft Excel by removing duplicate entries, correcting inconsistent text responses (for example, standardising "mtn", "MTN", and "Mtn"), handling missing values, and formatting the data into structured tables suitable for Pivot Table analysis.

3.4 User Reluctance Assessment and Mitigation

Three potential barriers to user adoption were identified: privacy concerns over the collection of location and network-usage data; battery-consumption concerns arising from continuous background monitoring; and data-usage concerns from periodic transmission of collected records. To mitigate these concerns, the design incorporated strong data encryption, clear privacy policies, user control over data sharing, optimisation of background collection intervals, compression and minimisation of transmitted data, an option to restrict synchronisation to Wi-Fi, and a simple, low-friction user interface.

3.5 Chapter Summary

The requirement gathering phase combined surveys, interviews, brainstorming, and reverse engineering to build a comprehensive, validated understanding of stakeholder needs. The resulting feature list and identified constraints formed the direct input to the requirement analysis presented in the next chapter.

CHAPTER FOUR: REQUIREMENT ANALYSIS

Requirement analysis examines, refines, validates, and organises the requirements collected in Chapter Three, ensuring the proposed system satisfies stakeholder expectations while remaining technically feasible, reliable, secure, and user-friendly.

4.1 Completeness of Requirements

The gathered requirements were reviewed and found to sufficiently cover three categories. Functional requirements include collecting user feedback, automatically gathering network metrics, storing data centrally, generating reports, operating continuously in the background, and synchronising data in real time. User requirements include a simple, intuitive interface, minimal battery consumption, protection of personal information, and meaningful insight for the user. Technical requirements include Android platform compatibility, internet connectivity, GPS/location services, cloud or local database integration, background service capability, and secure data transmission.

4.2 Clarity of Requirements

Several vaguely stated requirements were refined into precise, measurable statements. For example, the broad statement "the app should monitor the network" was refined to: the application shall automatically monitor and record network parameters such as signal strength, latency, jitter, packet loss, network type, and timestamp information in real time. Similarly, "users should provide feedback" was refined to: the application shall periodically prompt users to rate network quality, call quality, internet speed, and overall satisfaction using a structured rating system.

4.3 Technical Feasibility Analysis

4.3.1 Mobile Platform Feasibility

Android was confirmed as a highly feasible platform choice given its wide adoption among subscribers, its rich set of network-monitoring APIs, and its mature support for background services and sensor integration.

4.3.2 Data Collection Feasibility

Automated collection of signal strength, network type, internet speed, GPS location, and timestamps was confirmed as achievable through the Android Telephony APIs, network monitoring libraries, and GPS services.

4.3.3 Database Feasibility

Candidate storage technologiesFirebase Realtime Database, MySQL, and SQLitewere assessed for security, real-time synchronisation, scalability, and accessibility, all of which were found to be adequate for the project's needs. This informed the final choice of Room/SQLite for local storage and MySQL for the remote database, detailed in Chapter Eight.

4.3.4 Performance and Security Feasibility

Continuous background monitoring raised concerns over CPU usage, battery drain, and data consumption, which were addressed through optimised collection intervals and lightweight monitoring techniques. Because the system collects potentially sensitive data such as location and device information, data encryption, transparent permission requests, and secure communication protocols were confirmed as necessary and feasible security measures.

4.4 Dependency Relationships

Requirement	Dependency
Real-time data synchronisation	Internet connectivity
Location-based monitoring	GPS permission
Network performance tracking	Android device APIs
Cloud data storage	Database server availability
User feedback submission	User interaction
Report generation	Stored and processed data

4.5 Inconsistencies, Ambiguities, and Missing Information

Survey data contained inconsistencies such as varying capitalisation of network provider names ("mtn", "MTN", "Mtn") and device names, which were resolved through standardised naming conventions and duplicate removal. Ambiguous responsesfor example, users rating network quality simply as "Poor" without specifying whether the issue concerned speed, signal, or call qualitywere addressed by introducing more detailed rating categories in later analysis. The review also identified several requirements missing from the initial gathering: user authentication mechanisms, data encryption and security policies, offline data storage support, notification management, error handling, and data backup and recovery. These were incorporated into the system design.

4.6 Requirement Prioritisation

Priority	Requirements
High	User feedback collection; network monitoring; data storage; background operation; privacy protection
Medium	GPS integration; notifications; data visualisation; cloud synchronisation
Low	Dark mode; theme customisation; personalised dashboards

4.7 Classification of Requirements

4.7.1 Functional Requirements

The application shall allow users to submit feedback, monitor network quality automatically, collect signal strength and latency data, store user and network data, generate reports and charts, synchronise collected data with a server, record timestamps and locations, and operate continuously in the background.

4.7.2 Non-Functional Requirements

Performance: the application should respond quickly and collect data efficiently while keeping battery consumption low. Security: user information must be protected, data transmission encrypted, and access permissions controlled. Reliability: the application should run continuously with minimal failures and minimal data loss. Usability: the interface should be simple and intuitive. Compatibility: the application should support multiple Android versions and screen sizes.

4.8 Validation of Requirements with Stakeholders

The analysed requirements were validated by reviewing survey findings with users, confirming technical feasibility with developers, and verifying privacy and usability concerns. Stakeholders agreed that the proposed system would improve network-quality monitoring, enhance customer-satisfaction analysis, and support informed decision-making by network operators.

4.9 Chapter Summary

The requirement analysis phase transformed raw stakeholder input into complete, clear, prioritised, and validated requirements, resolving inconsistencies and filling gaps identified during review. These requirements were subsequently formalised in the Software Requirements Specification presented in Chapter Five.

CHAPTER FIVE: SOFTWARE REQUIREMENTS SPECIFICATION

This chapter presents the formal Software Requirements Specification (SRS) developed from the validated requirements in Chapter Four. It serves as the authoritative reference guiding the design and implementation described in the remaining chapters.

5.1 Overall Description

The proposed system is a mobile application that combines active user feedback with passive network monitoring. It operates in the background and periodically prompts users for feedback on their network experience, while automatically collecting network and device parameters such as signal strength, network type, internet speed, packet loss, latency, jitter, device information, GPS location, and timestamp. The collected data is stored in a centralised database where it can be analysed and visualised using charts and reports.

5.2 System Features

- User Feedback Collection: rate network quality, report call drops, evaluate internet speed and service reliability.
- Automatic Network Monitoring: collect signal strength, network type, latency, packet loss, internet speed, and device details.
- Background Operation: run continuously with minimal interruption to normal device usage.
- Data Storage: securely store collected data for future analysis.
- Data Visualisation: generate charts, pivot tables, graphs, and statistical summaries.
- Report Generation: produce network analysis, QoE evaluation, and performance monitoring reports.

5.3 Functional Requirements

5.3.1 User Registration and Identification

The system shall allow user identification, store basic device information, and associate collected data with users, anonymously where necessary.

5.3.2 Feedback Collection

The system shall prompt users for feedback periodically, allow users to rate network quality, record satisfaction levels, and capture call-drop experiences.

5.3.3 Network Monitoring

The application shall automatically monitor signal strength, network type, internet speed, latency, packet loss, and jitter.

5.3.4 Location Tracking

The system shall record user location during data collection and associate QoE data with geographical locations.

5.3.5 Data Storage, Reporting, and Background Services

The system shall store collected information securely with timestamps organised into structured tables, generate reports and visual analytics automatically, and operate continuously in the background while minimising resource consumption.

5.4 Non-Functional Requirements

Category	Requirement
Performance	Operate efficiently in real time; minimise battery and data consumption; respond quickly to user interactions
Security	Protect user privacy; encrypt sensitive data; use secure communication protocols; prevent unauthorised access
Reliability	Operate continuously with minimal failures; prevent data loss; recover gracefully from errors
Usability	Provide a simple interface; be easy to navigate; require minimal technical knowledge
Compatibility	Support Android devices across multiple versions and screen sizes

5.5 System Architecture Overview

The system architecture is organised into four layers. The Mobile Application Layer handles user interaction, data collection, background monitoring, and device communication. The Database Layer handles data storage, retrieval, organisation, and management. The Analytics Layer handles data processing, QoE analysis, chart generation, and statistical reporting. The Administration Layer handles monitoring of system performance, management of collected data, and report generation.

5.6 Data Requirements

Data Type	Description
User Feedback	User satisfaction ratings
Signal Strength	Network signal measurements
Network Type	2G, 3G, 4G, 5G
Location Data	GPS coordinates
Device Information	Device model and OS
Timestamp	Date and time of collection
Internet Performance	Speed, latency, jitter

5.7 Security Requirements

To ensure data privacy and confidentiality, the system implements user permission management, secure authentication mechanisms, data encryption, secure database access, and secure network communication, in compliance with applicable data-protection regulations.

5.8 Software and Hardware Requirements

Development tools used include Android Studio, Kotlin, MySQL, Microsoft Excel, and Google Forms. Hardware requirements include an Android smartphone with GPS capability, an internet connection, and server or cloud storage infrastructure for the backend.

5.9 Constraints, Assumptions, and Dependencies

The system's operation is constrained by dependence on internet connectivity, device compatibility, battery consumption, GPS accuracy, and potential user reluctance toward data sharing. It assumes that users possess Android smartphones, grant the necessary permissions, and have GPS and internet services available, and it depends on Android APIs, database availability, and cloud synchronisation services.

5.10 Use Case Descriptions

Use Case 1: Submit User Feedback

Actor: Mobile Subscriber. The user opens the application, the feedback form is displayed, the user submits ratings, and the system stores the feedback.

Use Case 2: Automatic Network Monitoring

Actor: System. The background service starts, the system collects network data, the data is timestamped, and the data is stored in the database.

Use Case 3: Generate Reports

Actor: Administrator/Operator. The administrator requests analysis, the system processes stored data, and charts and reports are generated.

5.11 Chapter Summary

This chapter has formalised the requirements of the QoETRACK system into a complete Software Requirements Specification, covering functional and non-functional requirements, system architecture, data requirements, security requirements, and representative use cases. This specification directly guided the system modelling, UI design, and database implementation presented in the following chapters.

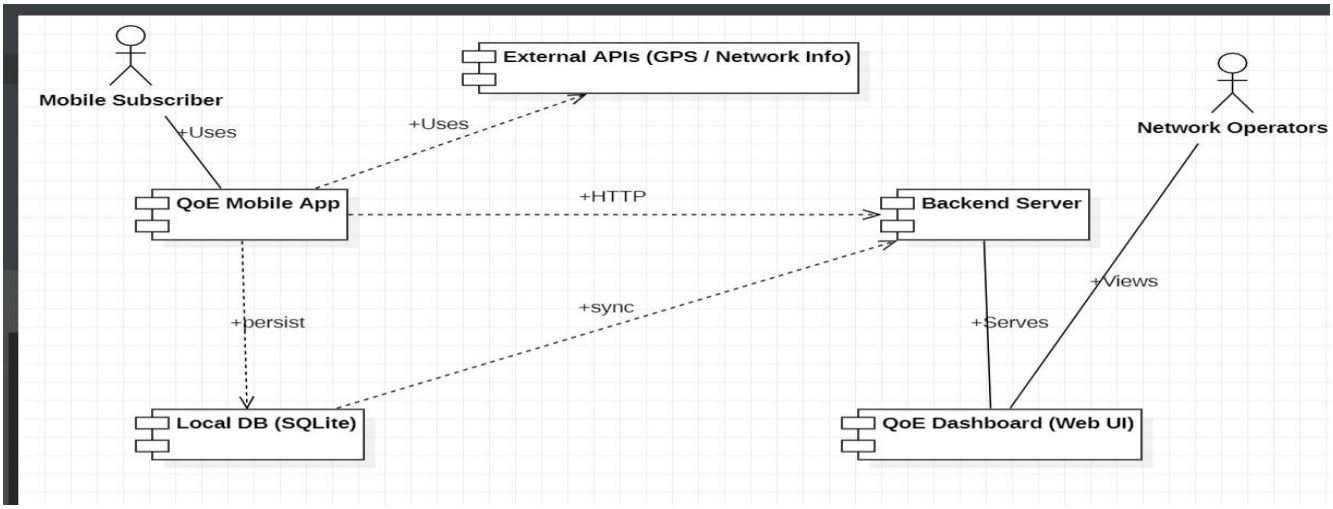
CHAPTER SIX: SYSTEM MODELLING AND DESIGN

This chapter presents a comprehensive system model for QoETRACK, engineered to collect Quality of Experience data in real time from mobile network subscribers in the Cameroonian mobile network landscape. The system design is articulated through six complementary UML and structured-analysis diagrams, each serving a distinct architectural purpose.

6.1 Context Diagram

The Context Diagram (Level-0 Data Flow Diagram) defines the boundary of the QoE Mobile App system and its interactions with external entities: the Mobile Subscriber, who provides feedback and grants permissions; Network Operators, who consume aggregated data via the dashboard; External APIs (GPS/Network Info), which supply signal strength, network type, and GPS coordinates; the Backend Server, a cloud-hosted REST API that receives and serves data; the Local DB (SQLite), used for offline buffering; and the QoE Dashboard, the web interface for operator analytics.

From	To	Data / Interaction
Mobile Subscriber	QoE Mobile App	Uses (provides feedback, grants permissions)
QoE Mobile App	External APIs	API request for signal/GPS data
QoE Mobile App	Local DB (SQLite)	Persist records locally
QoE Mobile App	Backend Server	HTTP upload of metric batches
Backend Server	QoE Dashboard	Serves aggregated data via HTTP
Network Operators	QoE Dashboard	Views analytics and reports



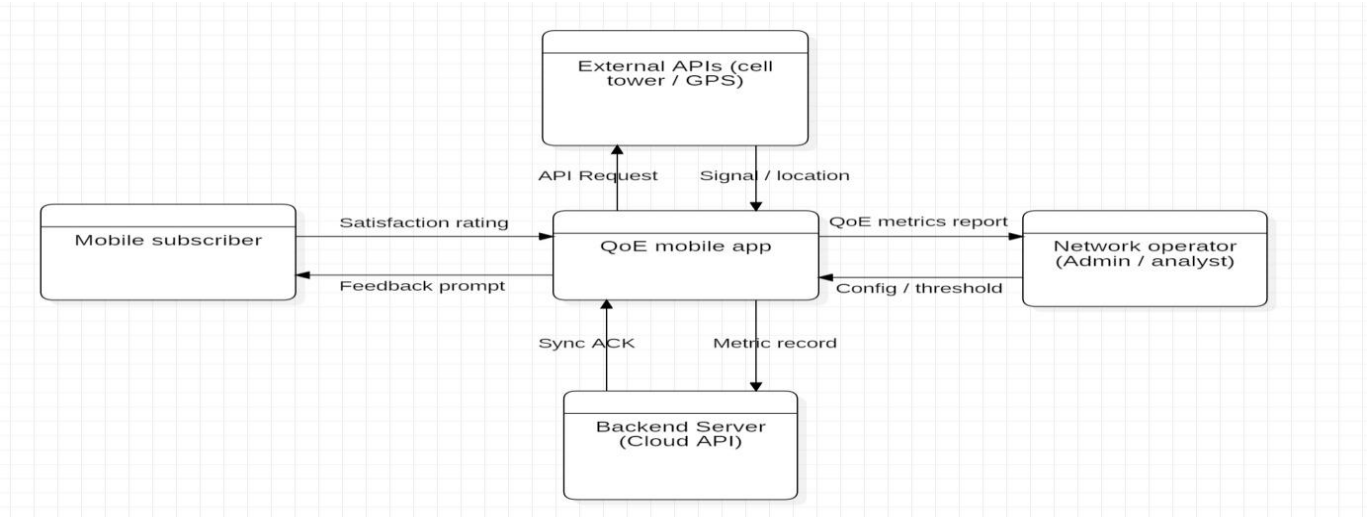
6.2 Data Flow Diagram (DFD)

Two levels of DFD were modelled. The Level-0 model identifies five processes: the Mobile Subscriber, sending ratings and receiving prompts; the QoE Mobile App, the central process hub; External APIs, returning signal and location data; the Backend Server, receiving uploads and returning sync acknowledgements; and the Network Operator, providing configuration and threshold inputs.

The Level-1 decomposition introduces four sub-processes and two data stores:

Process ID	Process Name	Description
P1	Metric Collector	Receives signal/GPS data from External APIs; writes metric records to DS1
P2	Feedback Capture	Prompts the subscriber, captures rating/consent, writes feedback records to DS1
P3	User Settings	Reads stored metrics, generates QoE reports, handles operator configuration
P4	Data Sync	Reads pending records from DS1, queues batches in DS2, uploads to Backend Server

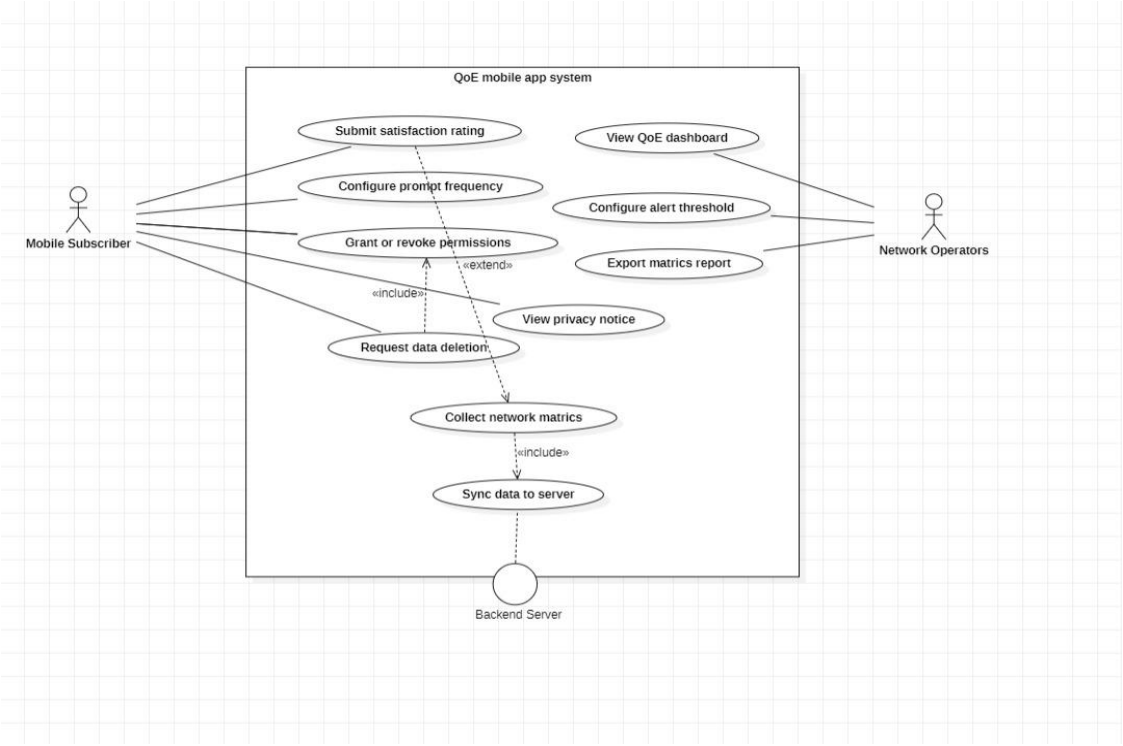
DS1 (Local SQLite DB) serves as the central on-device repository for raw metric and feedback records, while DS2 (Upload Queue) is a temporary staging store for batches pending synchronisation.



6.3 Use Case Diagram

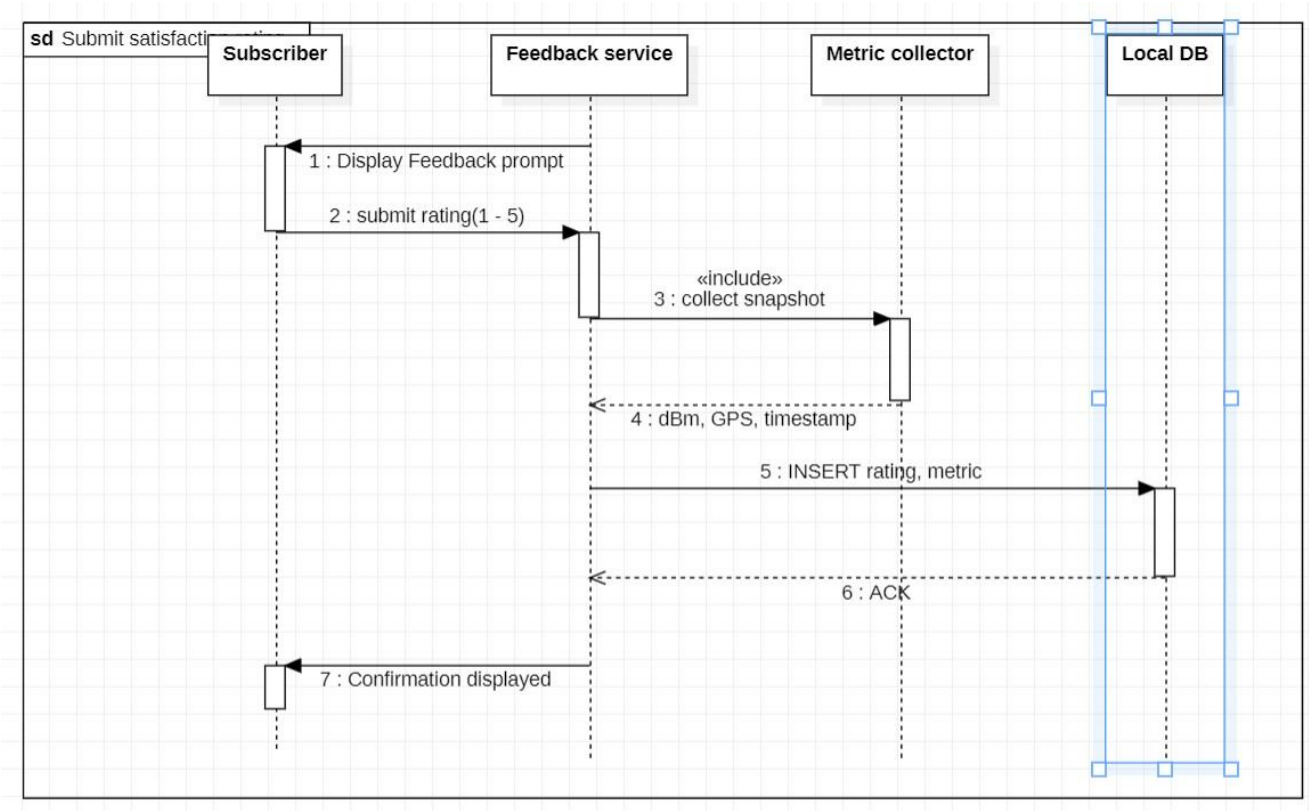
The Use Case Diagram defines the system boundary and its functional interactions for two actor types: the Mobile Subscriber and the Network Operator.

Use Case	Actor(s)	Description
Submit Satisfaction Rating	Mobile Subscriber	Rate current network experience on satisfaction, speed, and usability
View Privacy Notice	Mobile Subscriber	Read the data collection privacy statement
Configure Prompt Frequency	Mobile Subscriber	Set how often the app requests feedback
Grant or Revoke Permissions	Mobile Subscriber	Control Android permissions for location and network access
Request Data Deletion	Mobile Subscriber	Trigger deletion of locally and remotely stored records
Collect Network Metrics	System (Background)	Autonomously collect signal, network type, GPS, and latency
Sync Data to Server	System (Background)	Batch and upload pending records to the backend
View QoE Dashboard	Network Operator	View aggregated charts, heatmaps, and time-series data
Configure Alert Thresholds	Network Operator	Set minimum acceptable metric values
Export Metrics Report	Network Operator	Generate and download a CSV or PDF report



6.4 Sequence Diagrams

Ten sequence diagrams were modelled, one for each use case, describing the time-ordered message exchanges between system lifelines. Key flows include: Submit Satisfaction Rating, in which the Feedback Service prompts the subscriber, captures the rating and a metric snapshot, and persists both to the Local DB; Collect Network Metrics (Background), in which a background timer fires every five minutes, the Metric Collector queries External APIs for signal strength, network type, and GPS coordinates, and inserts the result into the Local DB; Sync Data to Server, in which the Sync Service reads unsynced records, batches them through the Upload Queue, transmits them via POST /metrics, and marks them as uploaded on receipt of a 200 OK response with sync_id; and Request Data Deletion, in which the app clears local records and sends a DELETE request to the backend before confirming to the subscriber.



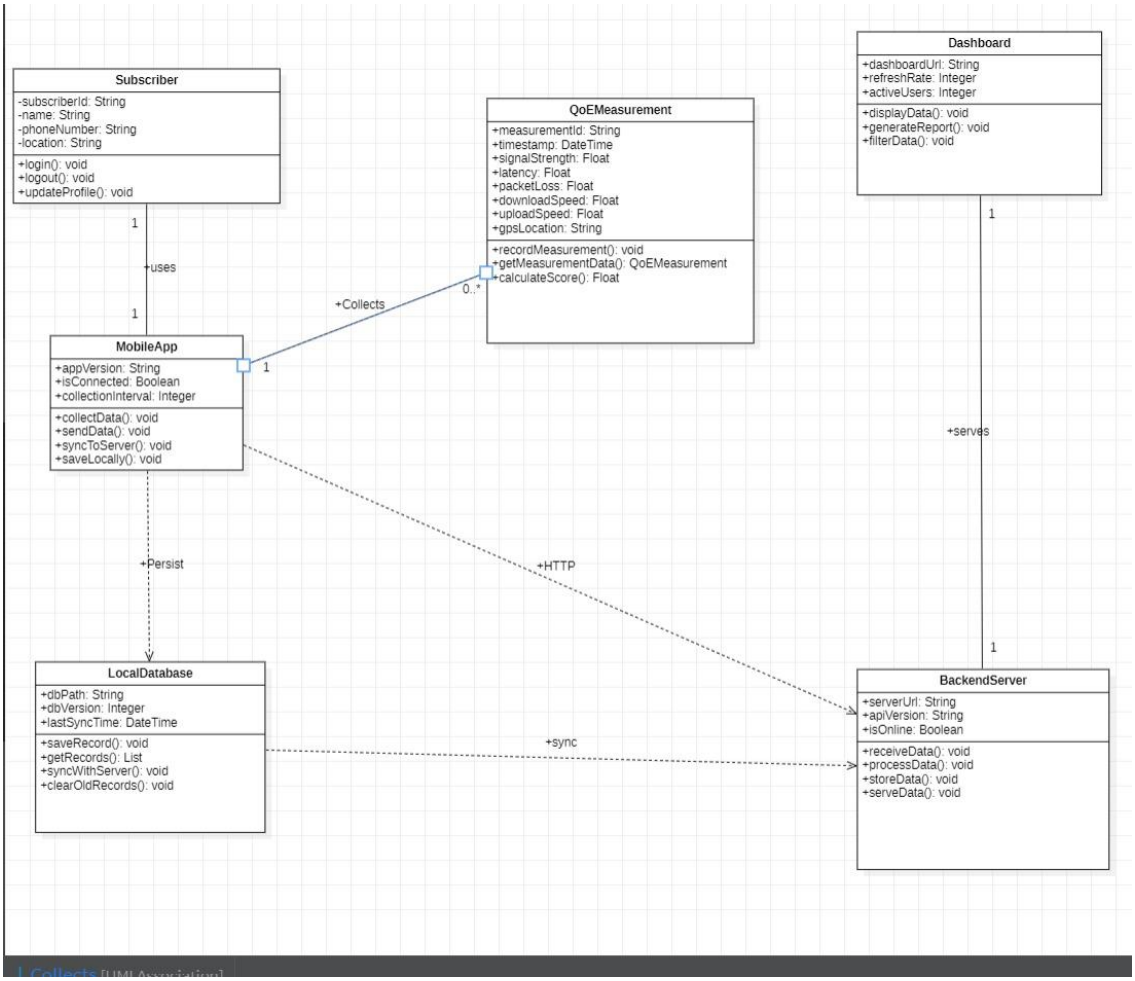
6.5 Class Diagram

The Class Diagram models the static structure of the system's software objects. Six core classes were identified: Subscriber, MobileApp, QoEMeasurement, LocalDatabase, BackendServer, and Dashboard.

Class	Key Attributes	Key Operations
Subscriber	subscriberId, name, phoneNumber, location	login(), logout(), updateProfile()

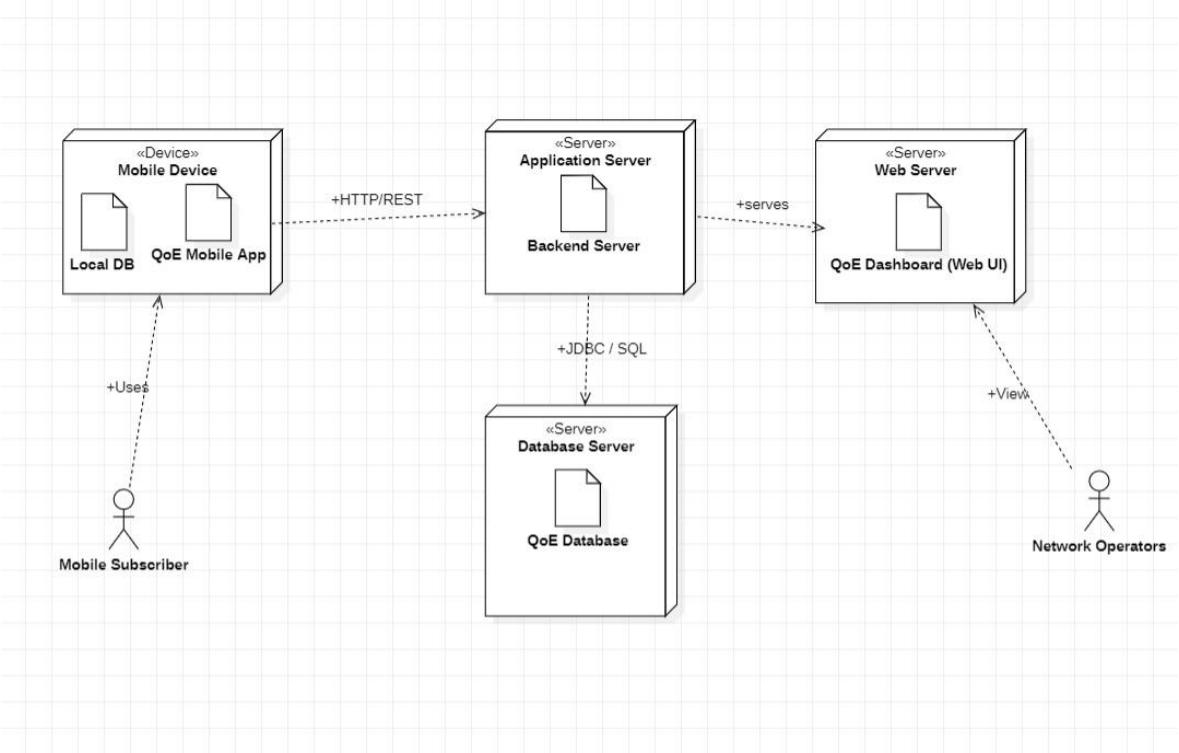
Class	Key Attributes	Key Operations
MobileApp	appVersion, isConnected, collectionInterval	collectData(), sendData(), syncToServer(), saveLocally()
QoEMeasurement	measurementId, timestamp, signalStrength, latency, packetLoss, gpsLocation	recordMeasurement(), calculateScore()
LocalDatabase	dbPath, dbVersion, lastSyncTime	saveRecord(), getRecords(), syncWithServer()
BackendServer	serverUrl, apiVersion, isOnline	receiveData(), processData(), storeData(), serveData()
Dashboard	dashboardUrl, refreshRate, activeUsers	displayData(), generateReport(), filterData()

Key relationships: Subscriber uses MobileApp; MobileApp collects QoEMeasurement; MobileApp persists to LocalDatabase and communicates with BackendServer over HTTP; LocalDatabase syncs to BackendServer; and BackendServer serves the Dashboard.



6.6 Deployment Diagram

The Deployment Diagram maps software artefacts to physical hardware nodes. The Mobile Device hosts the QoE Mobile App and the Local DB (SQLite). The Application Server hosts the Backend Server, connecting to the Database Server via JDBC/SQL. The Database Server hosts the centralised QoE Database (MySQL). The Web Server serves the QoE Dashboard to operators over a browser. Communication links are HTTP/REST between the Mobile Device and Application Server, JDBC/SQL between the Application Server and Database Server, and HTTP between the Application Server and Web Server.



6.7 Design Rationale

The QoE data collection system follows a three-tier client-server architecture: a mobile client tier (Android app and SQLite), an application server tier (REST API backend), and a presentation tier (web dashboard). This separation of concerns keeps the mobile app lightweight and functional offline while centralising storage and analytics on the server. Key design decisions include an offline-first architecture with local buffering and an upload queue, background collection every five minutes, privacy-by-design through dedicated permission and deletion flows, role-based access separating subscriber and operator interactions, and RESTful communication throughout for scalability and ease of integration.

6.8 Chapter Summary

The six diagrams presented in this chapter Context, DFD, Use Case, Sequence, Class, and Deploymentform a unified architectural blueprint guiding the system from user interaction through data collection, local storage,

server-side processing, and operator-facing analytics. This blueprint was directly realised in the user interface and database implementation described in Chapters Seven and Eight.

CHAPTER SEVEN: USER INTERFACE DESIGN AND IMPLEMENTATION

Building directly on the system architecture established in Chapter Six, this chapter translates the UML-defined componentsthe Mobile Subscriber actor, the QoE Mobile App system boundary, and the identified use casesinto a functional, visually branded Android user interface. The work is organised around three deliverables: App Identity, Visual Design, and Frontend Implementation.

7.1 App Identity

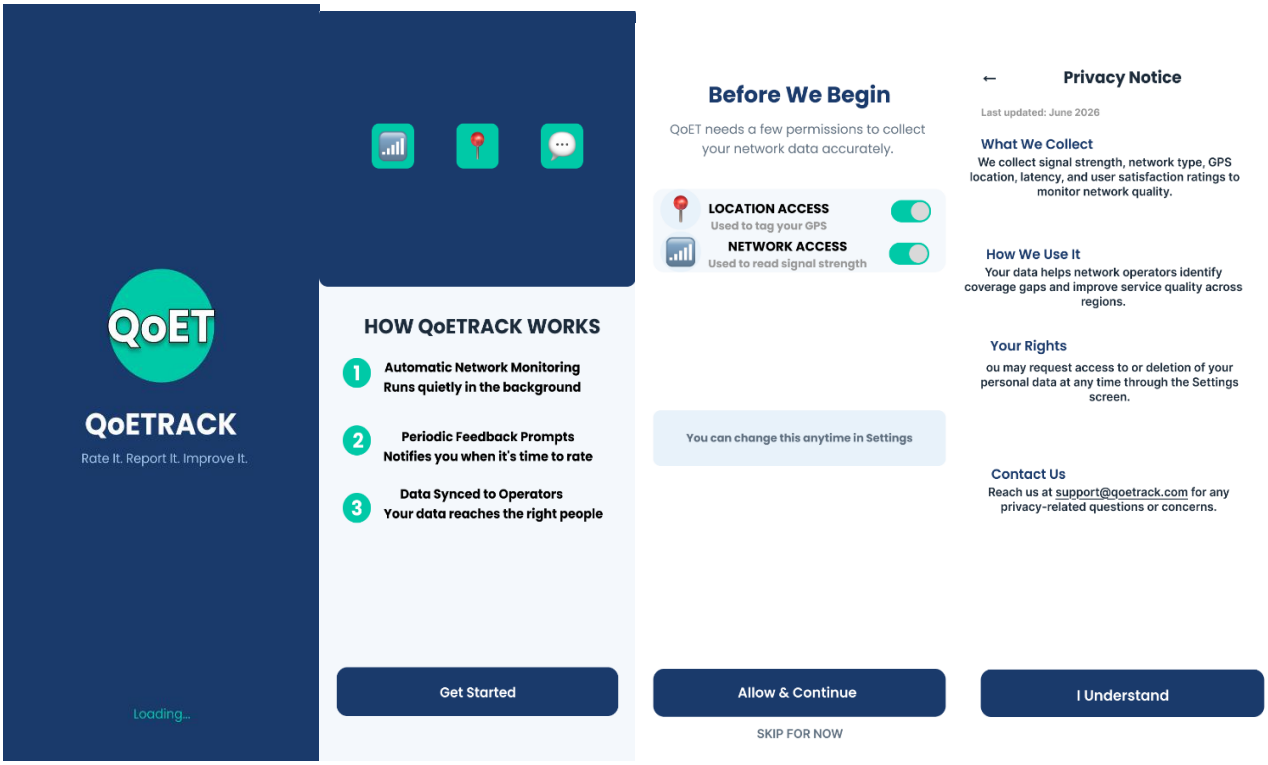
The application was named QoETRACK, a compound of QoE (Quality of Experience) and Track, with the tagline "Rate It. Report It. Improve It." The brand colour palette uses Navy (#1A3A6B) as the primary colour for top bars, hero cards, and buttons; Teal (#00C9A7) as an accent for the logo, step circles, and badges; a light Background (#F4F8FC); white Surface for cards; dark text for body copy; muted grey for captions; and red for the destructive Request Data Deletion action. Two typefaces were selected: Poppins (Bold, SemiBold, Regular) for headings and metric values, and Inter (Regular) for body text and captions. The logo consists of a circular teal shape containing the text "QoET" paired with the full app name beneath it.

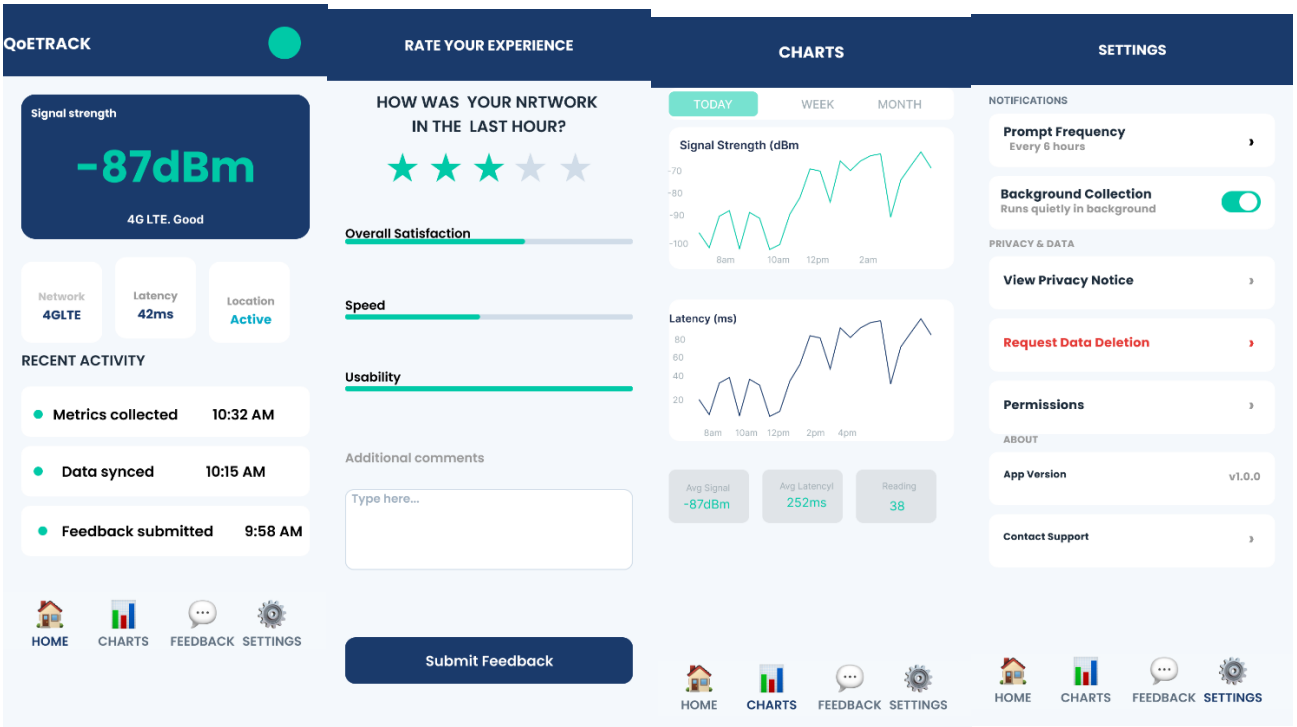
7.2 Visual DesignFigma Mockups

All seven screens were designed in Figma using an Android Small frame (360 × 800px), with colour styles, text styles, and reusable components applied consistently and linked into a clickable prototype flow.

Screen	Task 4 UML Reference	Description
Splash Screen	Context Diagram (QoE Mobile App)	Navy gradient background, centred circular teal logo, app name and tagline, animated loading bar; auto-navigates after 2.6 seconds
Onboarding Screen	View Privacy Notice; Sequence Diagram 4	Dark navy banner with three feature icons, scrollable feature list, and a Get Started button
Permission Request Screen	Grant/Revoke Permissions; Sequence Diagram 3	Two permission cards with toggle switches, an informational note, and Allow & Continue / Skip options
Home Dashboard Screen	Collect Network Metrics; DFD P1	Live dBm hero card, three metric mini-cards (Network, Latency, Location), Recent Activity list, and Sync Now button
Feedback Form Screen	Submit Satisfaction Rating; Sequence Diagram 1	5-star rating, three labelled sliders, comments field, and Submit Feedback button

Screen	Task 4 UML Reference	Description
Settings Screen	Configure Prompt Frequency; Request Data Deletion	Notifications, Privacy & Data, and About sections with prompt frequency, deletion, and permissions rows
Privacy Notice Screen	View Privacy Notice; Sequence Diagram 4	Full data collection policy in formatted sections with an I Understand button





7.3 Frontend Implementation

The frontend was implemented in Android Studio using Kotlin and XML-based view layouts, with the Material Components library for theming, and ConstraintLayout / LinearLayout as the primary containers. The application targets a minimum SDK of API 24 (Android 7.0) and a target SDK of API 37 (Android 15), using the Theme.AppCompat.Light.NoActionBar theme and the constraintlayout, cardview, material, and appcompat dependency libraries.

7.3.1 Splash Screen

Implemented as SplashActivity.kt with activity_splash.xml, using a navy gradient background and a circular teal logo drawn with an oval shape drawable. SplashActivity manages a five-stage entrance animation using ObjectAnimator and OvershootInterpolator, before checking a SharedPreferences flag (onboarding_complete) to route first-time users to Onboarding and returning users directly to the Dashboard.

7.3.2 Onboarding Screen

Implemented as OnboardingActivity.kt with activity_onboarding.xml, using a vertical LinearLayout with layout_weight to split the screen 40/60 between the dark banner and scrollable contenta deliberate design decision after an earlier ConstraintLayout percentage-height approach failed to render correctly. A staggered entrance animation reveals the banner, title, and feature rows at 150ms intervals, and the Get Started button persists the onboarding_complete flag before navigating to the Dashboard.

7.3.3 Dashboard Screen

Implemented as DashboardActivity.kt with activity_dashboard.xml, using a ConstraintLayout root with a fixed top bar, a scrollable content area, and a fixed bottom navigation bar. The updateSignal() method accepts dBm, network type, and latency values and colours the signal reading teal, amber, or red according to quality; a simulated update cycle refreshes the display every five seconds.

7.4 Navigation and State Management

The application comprises seven screens. On first launch it follows the flow Splash → Onboarding → Permission → Dashboard; on subsequent launches it follows the shortened flow Splash → Dashboard. From the Dashboard, users navigate via the bottom navigation bar to Feedback and Settings, and from Settings to Privacy Notice and Permission. Both SplashActivity and OnboardingActivity declare android:noHistory="true" so they cannot be revisited via the back button, and the onboarding_complete flag in SharedPreferences (qoetrack_prefs) governs routing on every subsequent launch.

7.5 Implementation Challenges and Resolutions

Several technical issues were encountered and resolved during implementation, summarised below.

Challenge	Resolution
Gradle IllegalDependencyNotation from Groovy-style strings in a Kotlin DSL project	Replaced with full implementation("group:artifact:version") format
Invalid compileSdk android-35.1 syntax	Replaced with compileSdk = 35
AAR dependency required compileSdk 35+	Bumped compileSdk and targetSdk to 35
android:hintTextColor / android:paintFlags not valid XML attributes	Applied programmatically via setHintTextColor() and Paint.UNDERLINE_TEXT_FLAG
Onboarding screen skipped due to a stale onboarding_complete flag	Uninstalled the app to reset SharedPreferences
Onboarding navigation not firing	Replaced instantiation-only call with navigateToPermission() inside withEndAction
Logo rendering as a square	Changed drawable shape from rectangle to oval
Font references failing	Created res/font/ directory and correctly renamed Poppins weight files

7.6 Screens Summary

Screen	Layout File	Activity File	Status
Splash	activity_splash.xml	SplashActivity.kt	Implemented & tested
Onboarding	activity_onboarding.xml	OnboardingActivity.kt	Implemented & tested
Permission	activity_permission.xml	PermissionActivity.kt	Implemented & tested
Dashboard	activity_dashboard.xml	DashboardActivity.kt	Implemented & tested
Charts	Activity_chart.xml	ChartActivity.kt	Implemented & tested
Feedback Form	activity_feedback.xml	FeedbackActivity.kt	Implemented & tested
Settings	activity_settings.xml	SettingsActivity.kt	Implemented & tested
Privacy Notice	activity_privacy.xml	PrivacyActivity.kt	Implemented & tested

7.7 Chapter Summary

Task 5 successfully translated the system model from Chapter Six into a functional, visually branded Android application. All seven screens were implemented and tested on a physical Android device, with the complete

navigation flow, animations, and brand styling functioning correctly, ready for integration with the database and backend layer described in Chapter Eight.

CHAPTER EIGHT: DATABASE DESIGN AND IMPLEMENTATION

This chapter documents the complete database design and implementation for QoETRACK. The database architecture is a two-tier system: a local Room/SQLite database embedded within the Android application for offline-first operation, and a remote MySQL database hosted on Railway cloud infrastructure, accessible through a RESTful Express.js backend API.

8.1 Data Elements

Four entities were identified, capturing every data element the application needs to store. The AppSettings entity is local-only and is not synchronised to the backend.

Entity	Key Fields	Description
User	user_id (PK), device_id (unique), created_at	Registered mobile subscriber, one registration per device
NetworkMetric	metric_id (PK), user_id (FK), network_type, signal_strength, latency, jitter, packet_loss, bandwidth, latitude, longitude, recorded_at, synced	A single passive data collection event recorded every 15 minutes
Feedback	feedback_id (PK), user_id (FK), overall_satisfaction, speed_rating, usability_rating, comments, submitted_at, synced	A subjective quality rating submitted through the Feedback screen
AppSettings*	id (PK = 1), dark_mode_enabled, bg_collection_enabled, prompt_frequency_hrs	Local-only singleton record storing user preferences

8.2 Conceptual Design

The User entity is the central entity to which all collected data is associated, keyed by an auto-incremented user_id with device_id ensuring one registration per device. NetworkMetric represents a passive collection event containing both objective network measurements and spatial context, carrying a synced flag to track upload status. Feedback represents a subjective rating submitted through the Feedback screen, also carrying a synced flag. AppSettings is a single-row, local-only entity storing UI preferences with no analytical value to network operators.

Three relationships were identified: a User generates many NetworkMetric readings (1:N); a User submits many Feedback entries (1:N); and a User has one local AppSettings record (1:1, Room only).

8.2.1 Key Design Decisions

- Offline-first architectureall data is written to local Room/SQLite first, then asynchronously pushed to MySQL via WorkManager, ensuring data is never lost due to network unavailability.
- Synced flag patternNetworkMetric and Feedback both carry a boolean synced column, set true only on a confirmed HTTP 201 response from the backend.
- Single-user modelthe current implementation uses one user registration per device (device_id as unique key), keeping the schema simple for the research context.
- Snake_case conventionconsistent column naming between Android Room entities, Sequelize models, and the MySQL schema.

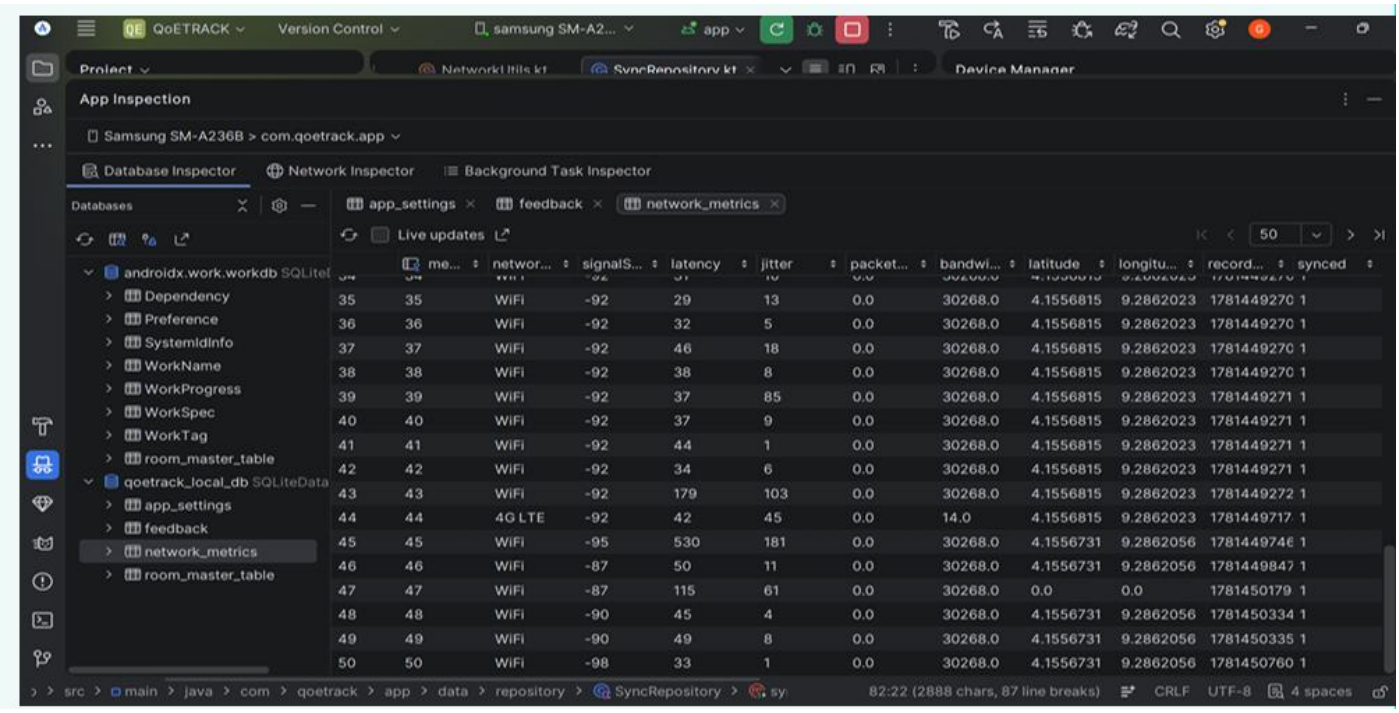
8.3 Entity Relationship Diagram

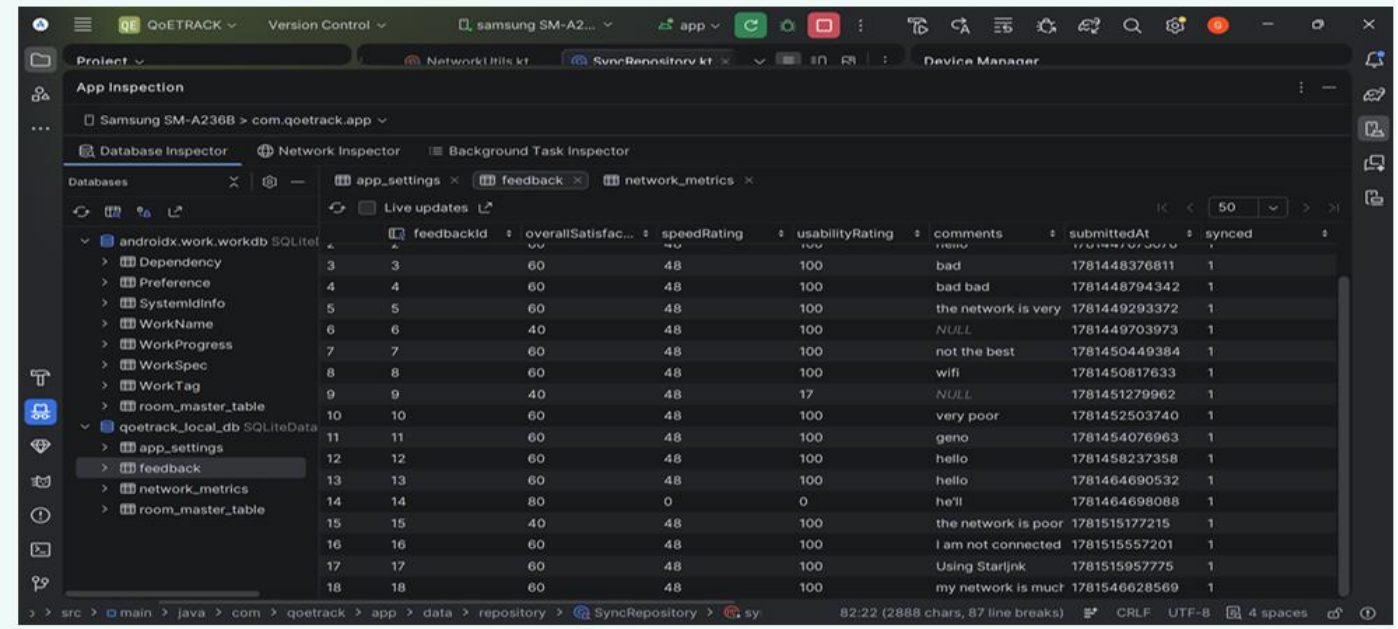
The ER diagram shows Users as the central/parent entity with user_id as primary key; NetworkMetric and Feedback as child entities with user_id as a foreign key configured with ON DELETE CASCADE; 1:N relationships from User to both child entities; and AppSettings as an isolated, local-only entity with no foreign key relationships.

8.4 Local DatabaseAndroid Room (SQLite)

Room is Google's abstraction layer over SQLite, providing compile-time SQL validation, LiveData/Flow integration, and type safety. The local database, named qoetrack_local_db, registers three entitiesNetworkMetricEntity, FeedbackEntity, and AppSettingsEntitythrough an abstract QoETrackDatabase class exposing a DAO for each. Each DAO defines the SQL operations required by the application, including inserting new records, querying unsynced rows, marking rows as synced, observing the latest reading reactively via Flow, and computing aggregate statistics such as average signal strength over a time window.

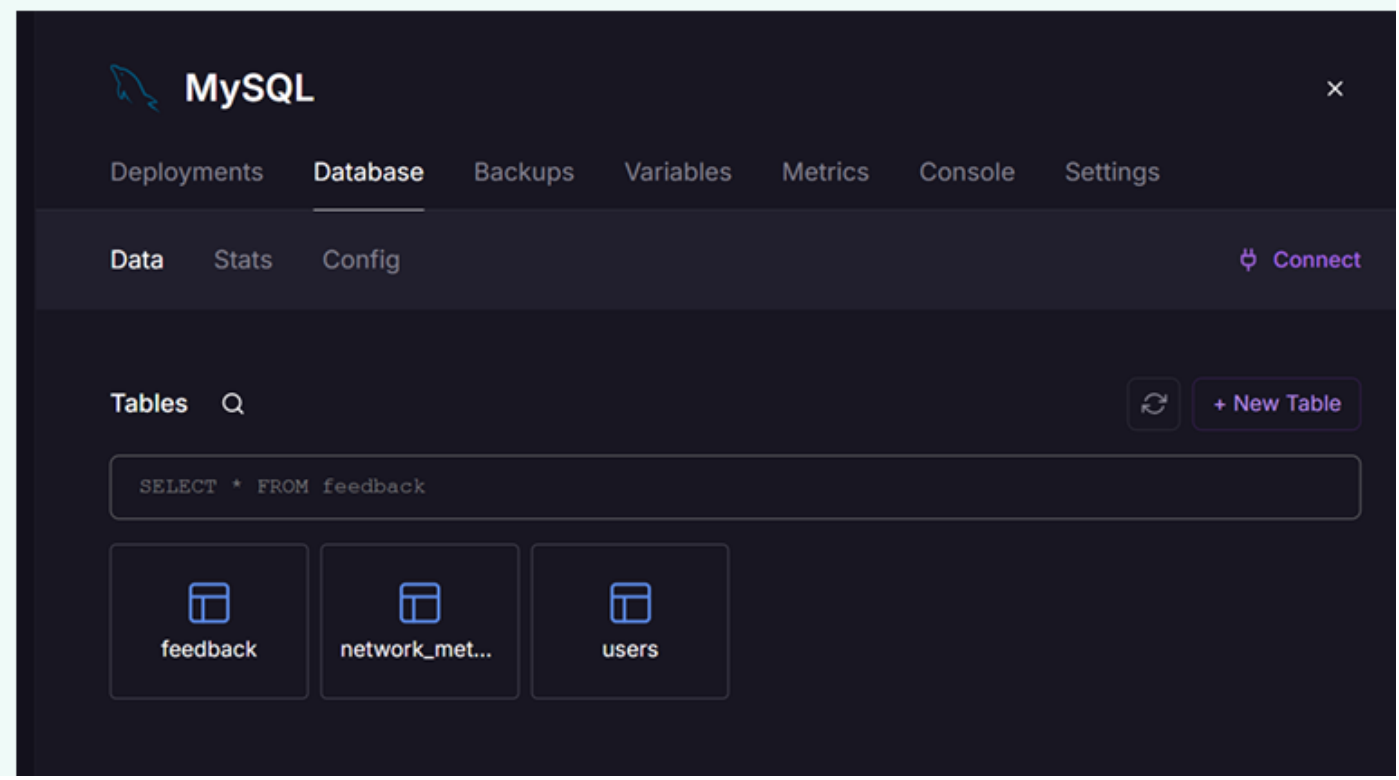
Verification using Android Studio's Database Inspector confirmed the local database was fully operational, showing over 50 real network_metrics rows collected from a physical device in Buea, Cameroon (latitude ≈ 4.1556, longitude ≈ 9.2862), together with 10 genuine user-submitted feedback rows.





8.5 Remote DatabaseMySQL on Railway

The remote database uses MySQL 8.0 hosted on Railway, mirroring the local schema through three tablesusers, network_metrics, and feedbackcreated automatically by Sequelize's sequelize.sync() on first backend deployment. Both network_metrics and feedback declare a foreign key on user_id referencing users(user_id), with ON DELETE CASCADE to preserve referential integrity. Railway's Database Inspector confirmed all three tables were created and populated successfully.

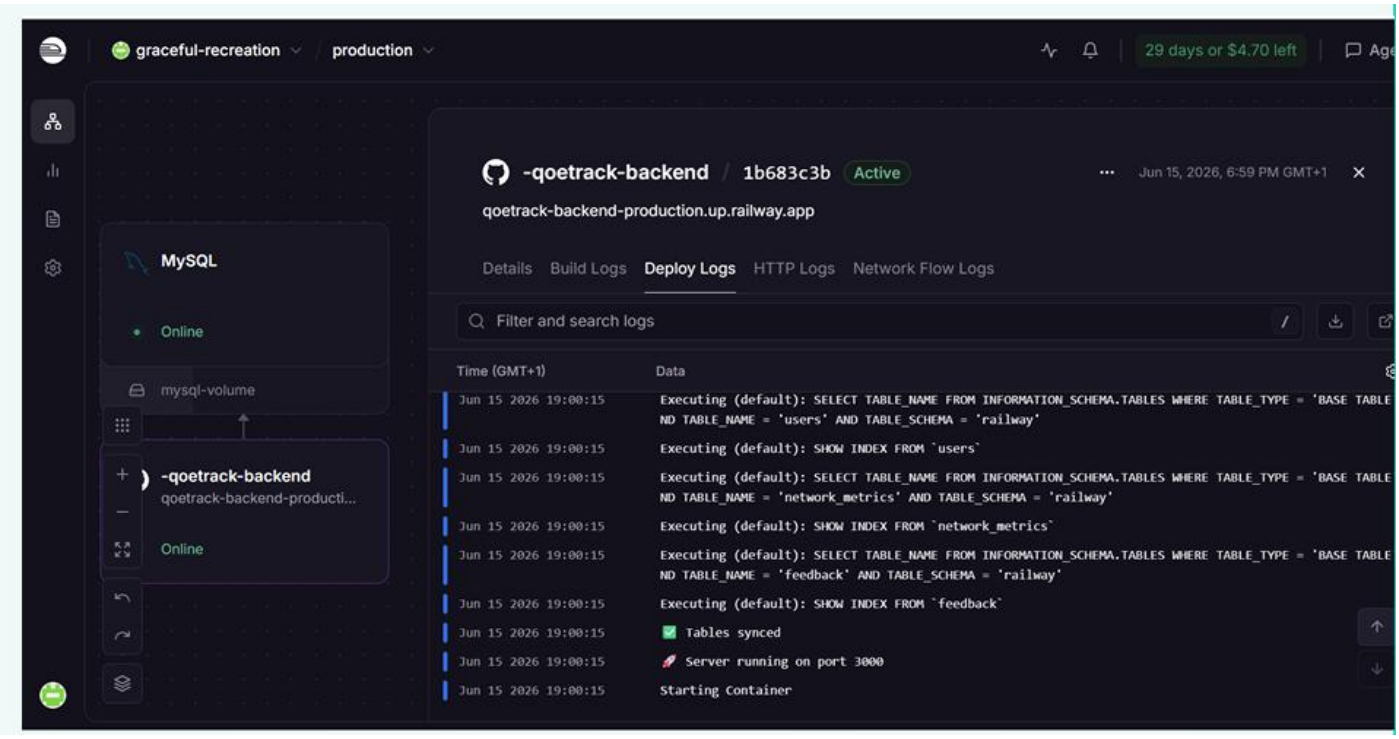


8.6 Backend Implementation

The backend API is implemented in Node.js using the Express.js framework and the Sequelize ORM with the mysql2 driver, deployed on Railway cloud infrastructure.

Method	Endpoint	Status	Description
GET	/	200 OK	Health check
POST	/api/metrics	201 Created	Receives a batch of network metrics from the Android app
GET	/api/metrics/user/:userId	200 OK	Returns the last 50 metric readings for a user
POST	/api/feedback	201 Created	Receives a feedback submission from the Android app
GET	/api/feedback/user/:userId	200 OK	Returns the last 50 feedback entries for a user

Each database table is represented by a Sequelize model mapping directly onto the MySQL schema, with a belongsTo association linking NetworkMetric and Feedback records to their owning User. Deployment logs confirmed a successful chain of database connection, table synchronisation, and server startup on port 3000, and the backend was reachable at qoetrack-backend-production.up.railway.app.

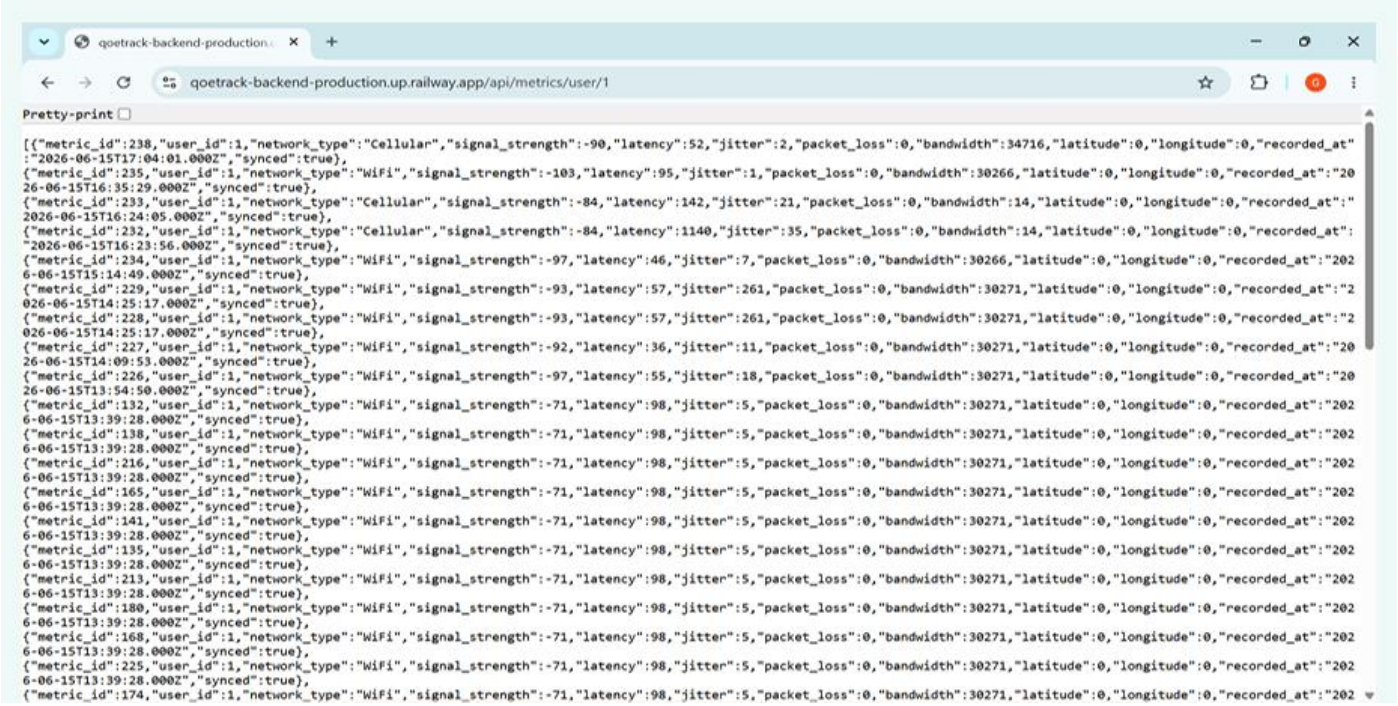


8.7 Connecting the Database to the Backend

The complete data pipeline runs from device sensor collection, through local persistence, to cloud storage. On the Android side, TelephonyManager and the FeedbackFragment feed a MetricCollectionWorker and FeedbackDao respectively, both of which persist first to Room before a SyncWorker uploads pending rows to the Express.js backend, which in turn writes to MySQL through Sequelize.

8.7.1 Retrofit HTTP Client

Retrofit 2 is configured with the base URL https://qoetrack-backend-production.up.railway.app/ and a Gson converter to serialise and deserialise JSON payloads between the Android client and the backend.



[{"metric_id":238,"user_id":1,"network_type":"Cellular","signal_strength":-90,"latency":52,"jitter":2,"packet_loss":0,"bandwidth":34716,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T17:04:01.000Z","synced":true},
{"metric_id":235,"user_id":1,"network_type":"WiFi","signal_strength":-103,"latency":95,"jitter":1,"packet_loss":0,"bandwidth":30266,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T16:35:29.000Z","synced":true},
{"metric_id":233,"user_id":1,"network_type":"Cellular","signal_strength":-84,"latency":142,"jitter":21,"packet_loss":0,"bandwidth":14,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T16:24:05.000Z","synced":true},
{"metric_id":232,"user_id":1,"network_type":"Cellular","signal_strength":-84,"latency":1140,"jitter":35,"packet_loss":0,"bandwidth":14,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T16:23:56.000Z","synced":true},
{"metric_id":234,"user_id":1,"network_type":"WiFi","signal_strength":-97,"latency":46,"jitter":7,"packet_loss":0,"bandwidth":30266,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T15:14:49.000Z","synced":true},
{"metric_id":229,"user_id":1,"network_type":"WiFi","signal_strength":-93,"latency":57,"jitter":261,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T14:25:17.000Z","synced":true},
{"metric_id":228,"user_id":1,"network_type":"WiFi","signal_strength":-93,"latency":57,"jitter":261,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T14:25:17.000Z","synced":true},
{"metric_id":227,"user_id":1,"network_type":"WiFi","signal_strength":-92,"latency":36,"jitter":11,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T14:09:53.000Z","synced":true},
{"metric_id":226,"user_id":1,"network_type":"WiFi","signal_strength":-97,"latency":55,"jitter":18,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:54:50.000Z","synced":true},
{"metric_id":132,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":138,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":216,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":165,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":141,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":135,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":213,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":180,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":168,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":225,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"2026-06-15T13:39:28.000Z","synced":true},
{"metric_id":174,"user_id":1,"network_type":"WiFi","signal_strength":-71,"latency":98,"jitter":5,"packet_loss":0,"bandwidth":30271,"latitude":0,"longitude":0,"recorded_at":"202

8.7.2 Sync Worker

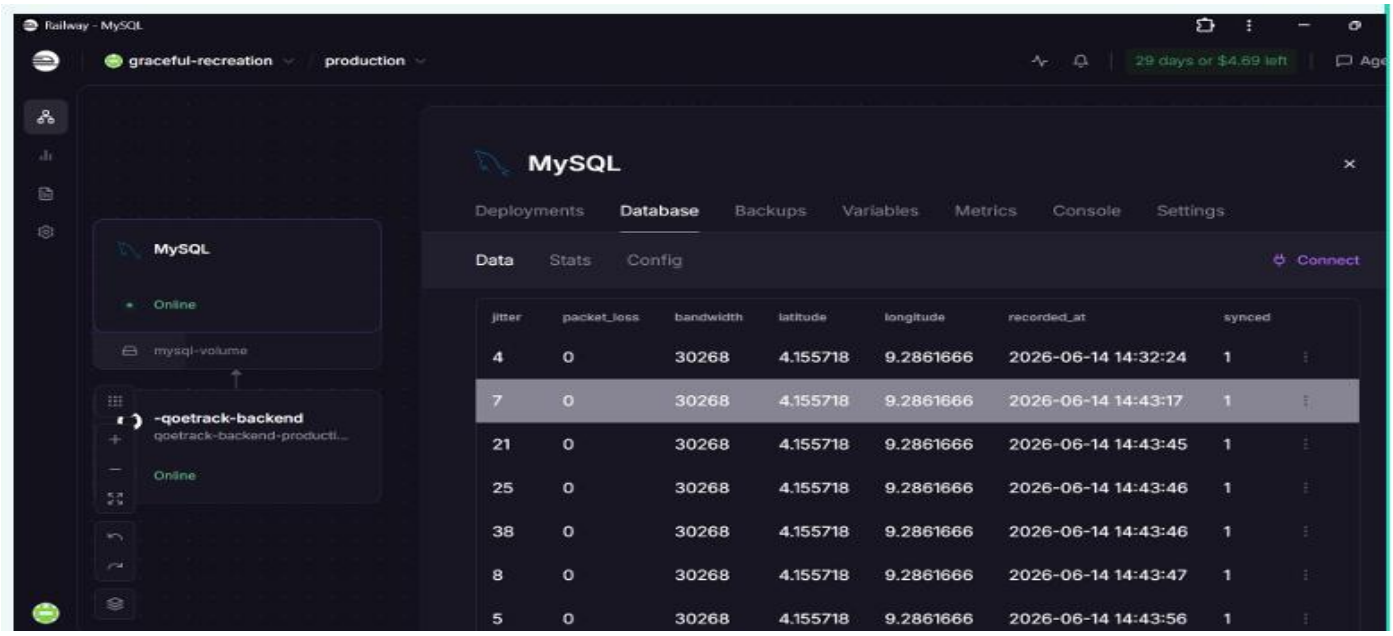
A WorkManager CoroutineWorker runs every 15 minutes, querying Room for unsynced rows through a SyncRepository, posting them to the backend via Retrofit, and returning Result.success() or Result.retry() depending on the outcome, providing automatic retry on failure.

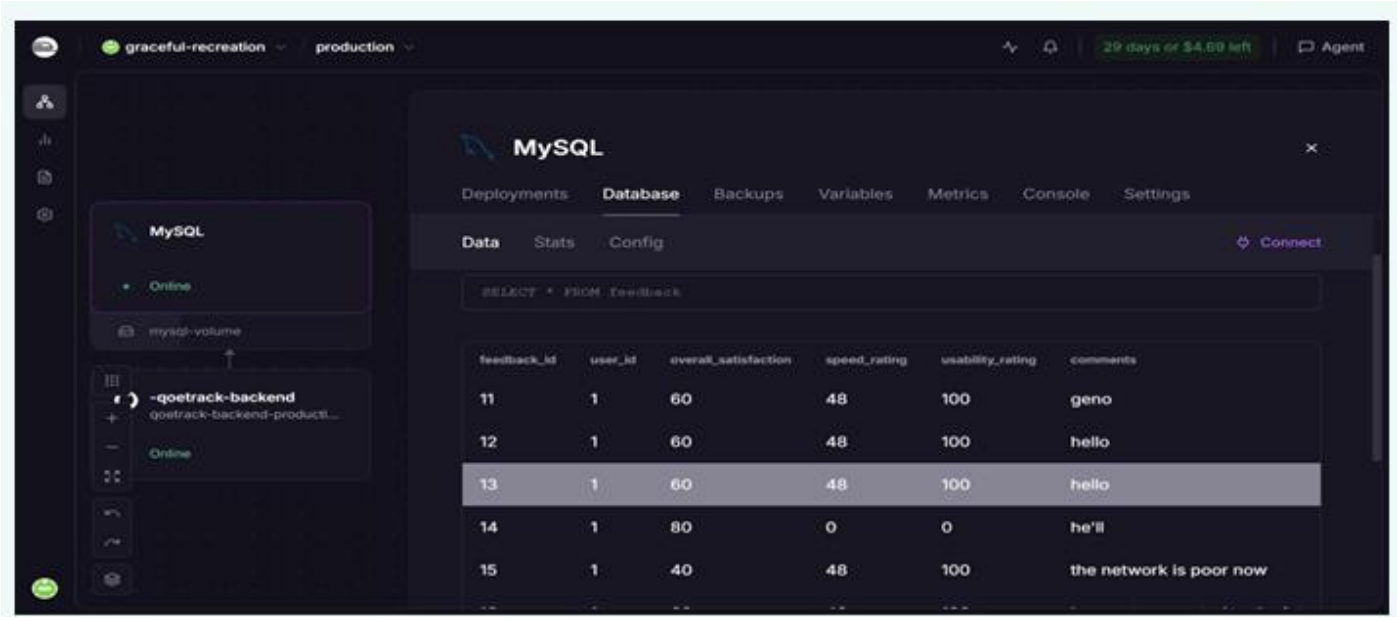
8.7.3 Sequelize Database Connection

The backend connects to Railway's MySQL instance using its internal hostname, with SSL enabled (rejectUnauthorized: false) as required for Railway-hosted MySQL.

8.8 End-to-End Sync Verification

The complete pipeline was verified end-to-end: Railway's MySQL Data tab showed network_metrics rows synced from the Android device with matching GPS coordinates (latitude ≈ 4.1556, longitude ≈ 9.2862) to those recorded locally, the feedback table showed synced rows with ratings and comments matching those entered on the device, and the Android Studio Database Inspector confirmed the synced column of local rows had been updated to 1 following a successful backend push.





8.9 Chapter Summary

8.9.1 Deliverables Summary

Sub-task	Status	Key Output
Data Elements	Complete	27 attributes across 4 entities documented
Conceptual Design	Complete	Entities, attributes, relationships, and design decisions defined
ER Diagram	Complete	Interactive ER diagram generated
Database Implementation	Complete	Room DB (local) + MySQL (Railway cloud), all 3 tables live
Backend Implementation	Complete	Express.js API deployed on Railway with 5 endpoints
Connecting DB to Backend	Complete	End-to-end sync confirmed: Room → Retrofit → Railway → MySQL

8.9.2 Limitations and Future Work

- Packet loss is currently estimated as 0.0; a proper implementation would require multiple ping attempts and failure counting.
- Authentication currently uses a fixed user_id = 1; a production system would require JWT/OAuth-based user authentication.
- AppSettings is local-only; future versions could synchronise settings across multiple devices for the same user.
- Rate limiting and API security (API keys, enforced HTTPS-only access) should be added before public release.

CHAPTER NINE: SUMMARY, CONCLUSION AND FUTURE WORK

9.1 Summary of Work Done

This report has documented the complete lifecycle of the QoETRACK project, from an initial review of mobile application development approaches, through structured requirement gathering and analysis, formal specification, UML-based system modelling, branded user interface design and frontend implementation, to a fully functional two-tier database and backend with verified end-to-end synchronisation. Each phase built directly on the artefacts produced in the phase before it, resulting in a coherent, traceable path from stakeholder need to working software.

9.2 Technical Achievements

- An offline-first architecture ensuring data is never lost even when the network is unavailable.
- Real Quality of Experience dataover 50 genuine network metric readings and 10 feedback entriescollected from a physical device in Buea, Cameroon during development and testing.
- A cloud-deployed backend, publicly accessible on Railway, exposing a documented REST API.
- Automated background synchronisation with retry logic implemented through WorkManager.
- Referential integrity enforced through cascading deletes across the local and remote schemas.
- A fully implemented and tested seven-screen Android application with consistent brand identity.

9.3 Limitations

As noted in Chapter Eight, the current implementation estimates packet loss rather than measuring it directly, relies on a single fixed user identifier rather than full authentication, keeps application settings local to a single device, and has not yet implemented production-grade API security such as rate limiting and enforced HTTPS-only access.

9.4 Recommendations for Future Work

- Implement proper multi-attempt packet-loss measurement for greater metric accuracy.
- Introduce JWT/OAuth-based authentication to support multiple registered users per device and account recovery.
- Extend settings synchronisation so preferences follow a user across multiple devices.
- Harden the backend with API keys, request rate limiting, and enforced HTTPS.

- Build out the operator-facing QoE Dashboard with heatmaps, regional filters, and exportable CSV/PDF reports as modelled in Chapter Six.
- Extend platform support to iOS to widen the subscriber base able to participate in data collection.

9.5 Conclusion

The QoETRACK project has successfully demonstrated that a lightweight, offline-first Android application can bridge the gap between network-side technical monitoring and the real Quality of Experience perceived by mobile subscribers in Cameroon. By combining passive metric collection with active user feedback, and by validating every layer of the systemfrom local storage to cloud databasewith real device data, the project fulfils the objectives set out in Chapter One and provides mobile network operators with a practical, extensible foundation for ground-truth, subscriber-side quality monitoring.

REFERENCES

- University of Buea, Faculty of Engineering and Technology, Department of Computer EngineeringCEF440: Internet Programming and Mobile Programming, Course Instructor: Dr. Valery Nkemeni, 2025/2026 Academic Year.
- Android Developers DocumentationRoom Persistence Library, WorkManager, and Telephony APIs.
- Sequelize DocumentationObject-Relational Mapping for Node.js.
- Express.js DocumentationFast, unopinionated, minimalist web framework for Node.js.
- Railway DocumentationCloud deployment platform.
- FigmaCollaborative interface design tool.
- Opensignal, Speedtest by Ookla, and nPerfreference network monitoring applications reviewed during requirement gathering.